

AI-Driven Personalized Learning and Remedial Recommendation

Step-by-step explanation of the project, the role of Gemini, Groq and OpenRouter, and how to answer mentor questions

How to use this guide. Read Part 1 for the whole idea in one minute. Part 2 explains why we use three AI models. Part 3 walks through every step of the project: what it does, why we do it, and a simple example. Part 4 gives the results. Part 5 lists the questions mentors usually ask, with answers you can say in your own words.

Part 1: The project in one minute

In a normal class, every student gets the same test and the same notes, even though each student is weak in different topics. Our system finds **exactly which concept** each student is weak in, gives that student **personal study material** for that concept, and then **tests them again** with fresh questions to check that they really improved.

It does this in a loop:

#	Stage	One-line meaning
1	Read the syllabus	The AI learns the course content (RAG).
2	Find the concepts	The course is split into 5 concepts, each linked to a Course Outcome (CO).
3	Measure the student	Past marks give a score per concept (OBE attainment).
4	Predict mastery	A deep-learning model (EKT) estimates how well the student knows each concept.
5	Find gaps	Any concept below the 70% target is a knowledge gap.
6	Remedial plan	Groq writes a personal study plan; OpenRouter and Gemini check it.
7	New questions	Groq writes MCQs; duplicates are removed; the other two AIs verify them.
8	Adaptive test	Easy, then Medium, then Hard; next level opens only after passing.
9	Repeat	New answers go back into the model; the loop continues until the student reaches the target.

One sentence for your viva: "Our system uses the course syllabus and the student's own answers to find the exact concepts they are weak in, generates a verified personal study plan and fresh verified questions using three free AI models, and re-tests the student adaptively until they reach the course outcome target."

The concepts used in our demo course (C Programming):

Code	Concept	Course Outcome
C1	Fundamentals and System Architecture	CO1
C2	Operators, Pointers and Control Structures	CO2
C3	Algorithmic Problem Solving and Array Manipulation	CO3
C4	String Manipulation and Text Processing	CO4
C5	Data Structures, Memory Management, and File Handling	CO5

Part 2: Why three AI models (Groq, OpenRouter, Gemini)?

A Large Language Model (LLM) can write questions and explanations, but it can also make mistakes ("hallucinate"): a wrong answer key, two correct options, or content that is not in the syllabus. If the same AI checks its own work, it usually repeats its own mistake. So we use **one AI to write** and **two different AIs from different companies to check**. This is called **cross-verification**. It is like a teacher setting a paper and two other teachers reviewing it.

Model	Role in our project	Model name we use	Why this one
Groq	Generator (writer) . Writes MCQs and remedial study plans.	openai/gpt-oss-120b	Very fast (special LPU chips), large free limit, strong 120-billion-parameter open model. Fast writing matters because the writer does the most work.
OpenRouter	Verifier 1 (checker) . Scores each question or plan from 1 to 5.	nvidia/nemotron-3-super-120b-a12b:free	One key gives access to many free models (":free"). A different company's model gives an independent second opinion.
Gemini (Google)	Verifier 2 (checker) . Also scores from 1 to 5.	gemini-flash-lite-latest	Free tier from Google; a third, independent model family. "Lite" has more free quota; "latest" does not get retired.

The **original paper** used OpenAI GPT as the writer and Gemini + DeepSeek as checkers. OpenAI and DeepSeek are paid. We kept the **same idea** (one writer, two independent checkers) but used **free** providers, so the project costs nothing to run. The code can still switch back to OpenAI/DeepSeek, or run fully offline with Ollama, just by changing `config.yaml`.

How a question is accepted

Step	What happens
1	Groq writes a question with 4 options, the correct answer, an explanation, difficulty and Bloom level.
2	OpenRouter gives a score 1-5 (correct? clear? matches the syllabus and level?).
3	Gemini gives its own score 1-5.
4	Average score 3 or more: the question is accepted . Below 3: it is rejected .
5	For remedial plans, a low score is sent back to Groq with the checkers' feedback and rewritten (up to 2 rounds).

Making free APIs reliable

Free APIs have limits (requests per minute), can be busy (error 503), and model names get retired (error 404). We built a safety layer called the **FallbackProvider**:

Problem	What our code does
Too many requests (429)	Pacing: sends at most N requests per minute (Gemini 8, Groq 25, OpenRouter 15). If still limited, it waits and retries , using the wait time the server suggests.
Server busy (502/503/504)	Retries automatically after a short wait.
Key invalid, no balance, model removed (401/402/404)	Circuit breaker: stops calling that provider at once (no point retrying) and switches to the offline mock so the app never crashes.
No key at all	Uses an offline mock model so the demo and all tests still run.
Model name retired	Command <code>python -m plrs models</code> lists the models your key can use, so you can update config.yaml.

Part 3: Every step of the project

Step 1: Upload the syllabus and build the knowledge base (RAG)

What happens: The course syllabus / notes are read, split into units, cut into small pieces (chunks), turned into number vectors (embeddings), and stored in a FAISS vector index.

Why we do it: An LLM does not know *our* syllabus. If we just ask it for questions it may go outside the course. RAG (Retrieval-Augmented Generation) finds the relevant syllabus pieces first and gives them to the AI, so every question and explanation is **grounded in our course**.

Technical detail (for mentors): Chunk size 1000 characters with 100-character overlap (so a sentence at a boundary is not lost). Embedding model: all-MiniLM-L6-v2 (384-dimension, small and fast). FAISS gives fast similarity search. Top-4 chunks are retrieved per request.

Simple example: When we need questions on "Strings", the system searches the index and pulls the chunks that talk about strings and the null character, and gives only those to Groq.

Step 2: Extract concepts and map them to Course Outcomes

What happens: Each unit of the syllabus becomes a concept (C1-C5) and is linked to a Course Outcome (CO1-CO5).

Why we do it: Marks per subject are too coarse. Saying "you got 55% in C Programming" does not tell the student what to study. Concept-level tracking tells them "you are weak in Strings". Linking to COs fits the college's Outcome-Based Education (OBE) system.

Simple example: Unit 4 "String Manipulation and Text Processing" becomes concept C4, mapped to CO4.

Step 3: Collect student answers and compute OBE attainment

What happens: Every answer a student gives is stored as a record: student, question, concept, correct or not. From these we compute the percentage score per concept.

Why we do it: This is the raw evidence of what the student knows. OBE rules then classify each concept.

Technical detail (for mentors): Threshold 50%, target 70% (about 20% above threshold). Below 50 = Beginner, 50-70 = Intermediate, 70 and above = Expert. Training data: 600 learners, 30 questions, 24,000 records (synthetic, generated to follow the paper's statistics).

Simple example: A student gets 3 of 8 string questions right: 37.5%, which is below the 50% threshold, so the level is Beginner.

Step 4: Predict concept mastery with Knowledge Tracing (EKT)

What happens: A deep-learning model reads the student's **whole answer history in order** and predicts, for each concept, the probability they will answer the next question correctly. This probability is the **mastery**.

Why we do it: A raw percentage treats an old mistake and a recent correct answer the same. Knowledge tracing understands that learning changes over time, and it also uses the **text of the question** (hard vs easy question), so its estimate is more accurate.

Technical detail (for mentors): EKT = Exercise-aware Knowledge Tracing: question text embedding + LSTM + attention over similar past exercises (EKTA). We compare it with DKT (Deep Knowledge Tracing, which ignores question text). Hyper-parameters: embedding 100, hidden 100, dropout 0.3, learning rate 0.001, batch 32, 12 epochs, 75/25 train-test split. We added an extra (auxiliary) loss on the mastery output so the mastery values are meaningful, not flat.

Simple example: A student failed the first 3 array questions but got the last 4 right. Raw score says 57%; EKT sees the improving trend and estimates higher mastery.

Step 5: Detect knowledge gaps

What happens: Every concept whose mastery is below the 70% target is marked as a knowledge gap.

Why we do it: We only spend AI effort on what the student actually needs. A strong concept does not get remedial work.

Simple example: Mastery: C1 82%, C2 74%, C3 61%, C4 38%, C5 55%. Gaps = C3, C4, C5 (C4 is the most urgent).

Step 6: Generate a personal remedial recommendation

What happens: For each gap, Groq writes a study plan using the syllabus chunks (RAG) and the questions the student got wrong. OpenRouter and Gemini score it; if the average is below 3, it is rewritten using their feedback.

Why we do it: Generic notes do not fix a specific gap. The plan is tailored to this student's concept, their mastery level, and their actual mistakes, and it is checked by two independent AIs so it is correct.

Technical detail (for mentors): Five fixed sections: 1) Learning Objectives, 2) Recommended Topics to Revise, 3) Remedial Explanation, 4) Practice Activities, 5) Concept Gap Rationale. Maximum 2 refinement rounds; minimum verifier score 3.

Simple example: For C4 at 38% mastery the plan explains that strings in C end with '\0', lists topics to revise (strlen, strcpy, character arrays), gives practice tasks, and says why this gap matters.

Step 7: Generate fresh MCQs for re-testing

What happens: Groq generates about 3 times the number of questions needed for each concept and level (easy, medium, hard). Near-duplicate questions are removed, and each remaining question is verified by OpenRouter and Gemini.

Why we do it: Re-using the same questions lets students memorise answers. Fresh, unique, verified questions test real understanding. Generating 3x lets us throw away duplicates and bad questions and still have enough.

Technical detail (for mentors): Duplicate check: cosine similarity of MiniLM embeddings; above 0.85 counts as a duplicate. Questions are saved in a question bank (memory) so the student is not shown the same question again. Each question also carries a Bloom's taxonomy level (Remember, Understand, Apply...).

Simple example: Need 5 easy string questions: generate 15, remove 4 that are almost the same, 2 fail verification, pick 5 from the remaining 9.

Step 8: Adaptive evaluation (Easy, then Medium, then Hard)

What happens: The student is tested concept by concept. The Medium level opens only after Easy is passed, and Hard only after Medium. The instructor sets the rule as "N and P": N questions per level, P% needed to pass.

Why we do it: A weak student is not discouraged by hard questions at the start, and a strong student proves real depth. The highest level passed gives a clear category.

Technical detail (for mentors): Categories: none passed = Not Competent, Easy = Beginner, Medium = Intermediate, Hard = Expert. Core course example "5 and 80"; optional course example "1 and 100".

Simple example: Rule "5 and 80": 5 questions per level, 4 of 5 needed. Student passes Easy (5/5) and Medium (4/5) but fails Hard (2/5), so the category is Intermediate.

Step 9: Continuous learning loop

What happens: The answers from the new test are added to the student's history and fed back into the EKT model. Mastery is updated, remaining gaps get new plans, and the cycle repeats.

Why we do it: Learning is not one-time. The loop keeps adapting until the student reaches the target, just like a tutor who checks again after each lesson.

Simple example: After round 1, C4 goes from 38% to 72% (gap closed) but C5 is still 64%, so round 2 focuses only on C5.

Step 10: Web application (frontend + API)

What happens: A FastAPI backend exposes the pipeline, and a single-page web frontend shows each student's concept mastery, gaps, remedial plans and adaptive tests.

Why we do it: Teachers and students need a simple screen, not a command line.

Simple example: Start with `python -m plrs serve` and open the browser: pick a learner, see weak concepts highlighted, read the plan, take the test.

Part 4: Results

Knowledge tracing (test data)

Model	AUC	Accuracy	F1	MAE (lower is better)
EKT (ours, uses question text)	0.827	0.748	0.783	0.331
DKT (baseline)	0.665	0.630	0.710	0.447

EKT is better on every metric because it uses the question text as well as right/wrong. AUC 0.5 is random guessing and 1.0 is perfect.

Learning gain after remediation (30 simulated students)

Concept	Pre	Post	Improvement
Fundamentals	71.1	86.2	+21.2%
Operators	72.8	78.6	+8.0%
Arrays	79.4	87.5	+10.2%
Strings	66.7	92.5	+38.7%
Data structures	67.8	88.1	+29.9%

Improvement = (post - pre) / pre x 100. The paired t-test is significant ($p < 0.05$) for Fundamentals, Strings and Data structures.

Be honest in the review: the students and the effect of studying are **simulated**. This shows that the pipeline works end to end. A real classroom study is future work. Mentors respect this honesty more than an over-claim.

Part 5: Mentor questions and how to answer them

Answer in short, confident sentences. If you do not know something, say what you do know and what you would check. Never guess numbers.

A. About the idea

Q: What problem does your project solve?

A: Normal teaching gives everyone the same material. Our system finds the exact concept each student is weak in, gives them a personal, verified study plan and new questions, and re-tests them until they reach the course outcome target.

Q: What is new compared to existing systems?

A: Most systems either predict marks or recommend generic content. We connect four things in one loop: concept mastery from knowledge tracing, syllabus-grounded AI content (RAG), cross-verification by independent AI models, and adaptive re-testing mapped to OBE course outcomes.

Q: Which paper is your base and what did you change?

A: Pradeesh et al., IEEE Access 2025. We implemented the full framework, replaced the paid models (OpenAI, DeepSeek) with free ones (Groq, OpenRouter, Gemini), added a reliability layer (pacing, retries, circuit breaker, offline mock), an auxiliary mastery loss for EKT, duplicate-free question memory, and a web frontend.

Q: What is OBE?

A: Outcome-Based Education: the course is judged by whether students achieve defined Course Outcomes. We use a threshold of 50% and a target of 70% attainment for each outcome.

B. About the AI models

Q: Why do you use three AI models instead of one?

A: One writes and two check. A model checking its own output tends to repeat its own mistakes. Models from different companies are trained differently, so they catch different errors. This reduces hallucination.

Q: Why Groq as the generator?

A: It is the fastest free option, so the writer (which makes the most calls) is not a bottleneck. It runs a strong 120-billion-parameter open model, gpt-oss-120b.

Q: Why OpenRouter and Gemini as verifiers?

A: They are free and from different providers, so the two opinions are independent. OpenRouter gives us an NVIDIA Nemotron model and Gemini is Google's model.

Q: What if the two verifiers disagree?

A: We take the average score. The item is accepted only if the average is at least 3 out of 5. For remedial plans, the feedback from both is sent back to Groq to rewrite the plan.

Q: What if an API is down or the free limit is reached?

A: The FallbackProvider paces requests per minute, waits and retries on 429/503, and switches off a provider on permanent errors like 401/402/404. If nothing works, an offline mock keeps the app running.

Q: Is student data sent to these companies? Is it safe?

A: Only the concept name, syllabus chunks and question text are sent, not student names. API keys are kept in a .env file that is not committed. For full privacy the system can run locally with Ollama.

Q: How do you stop the AI from generating wrong answers?

A: Three layers: RAG gives it only our syllabus content, a fixed JSON format forces 4 options plus one answer index, and two independent verifiers must score it at least 3 out of 5.

Q: What is a hallucination?

A: When an LLM produces confident but wrong or made-up content. RAG and cross-verification are our defences.

C. About RAG

Q: What is RAG and why do you need it?

A: Retrieval-Augmented Generation. Before asking the AI, we retrieve the relevant parts of our syllabus and include them in the prompt. This keeps questions inside our course and reduces hallucination.

Q: Why a chunk size of 1000 and an overlap of 100?

A: 1000 characters is about one paragraph: enough context but small enough to be precise. The 100-character overlap stops a sentence from being cut and lost at a chunk boundary. These values come from the paper.

Q: What is an embedding? What is FAISS?

A: An embedding turns text into a list of numbers so similar meanings are close together. We use all-MiniLM-L6-v2. FAISS is a library from Meta that finds the closest vectors very quickly.

D. About knowledge tracing

Q: What is knowledge tracing?

A: Modelling a student's knowledge over time from the sequence of their answers, to predict whether they will answer the next question correctly.

Q: Difference between DKT and EKT?

A: DKT only uses concept and right/wrong. EKT also reads the question text and uses attention over similar past questions, so it knows a hard question is different from an easy one. In our results EKT reached an AUC of 0.827 against 0.665 for DKT.

Q: What is AUC?

A: Area Under the ROC Curve. It is the chance that the model gives a higher score to a correct answer than to a wrong one. 0.5 means random, 1.0 means perfect.

Q: Why an LSTM?

A: Answers come in order and learning depends on history. An LSTM remembers long sequences without the vanishing-gradient problem of a plain RNN.

Q: What is the auxiliary mastery loss you added?

A: With only the next-answer loss, the mastery output could stay nearly flat. We added a second loss that ties the mastery for each concept to the student's actual performance, so the mastery numbers are meaningful and can be used to find gaps.

Q: Where did the data come from?

A: Synthetic data that follows the statistics in the paper: 600 learners, 5 concepts, 30 questions and 24,000 records. Real student data needs institutional permission, so testing on real data is future work.

Q: How do you prevent overfitting?

A: Dropout of 0.3, a 75/25 train-test split, and watching train vs test AUC across epochs. The two curves stay close, so the model is not overfitting.

E. About questions and evaluation

Q: How do you avoid duplicate questions?

A: We embed each new question and compare it with the questions already in the bank. If the cosine similarity is above 0.85 it is a near-duplicate and is dropped. The bank remembers which questions each student has seen.

Q: Why generate 3 times more questions than needed?

A: Some are dropped as duplicates and some fail verification. Generating 3 times as many still leaves enough good questions.

Q: What is the "N and P" rule?

A: The instructor sets N questions per level and P% to pass. For example, "5 and 80" for a core course means 4 of 5 correct are needed before the next level opens.

Q: Why adaptive Easy, then Medium, then Hard?

A: It matches Bloom's taxonomy, from remembering up to applying. Weak students are not overwhelmed, and the highest level passed gives a clear category from Not Competent to Expert.

Q: How do you measure the quality of the generated questions?

A: Semantic similarity to the syllabus, readability (Flesch-Kincaid grade), diversity (Self-BLEU) and the verifier relevance score from 1 to 5.

F. About results, limits and future work

Q: How do you prove students actually improved?

A: We compare pre and post scores per concept and apply a paired t-test. With 30 simulated students every concept improved, and the gain is significant for three of the five concepts.

Q: Your students are simulated. Is the result valid?

A: It validates that the full pipeline works end to end and behaves as expected. It is not proof of real classroom impact. A pilot with real students is our next step. The original paper reported significant gains on all five concepts with real students.

Q: What are the limitations?

A: Synthetic data. Dependence on free API limits and model names that change. Only MCQs, not descriptive answers. Remediation does not yet look at which wrong option the student chose.

Q: What is your future work?

A: Real classroom trials. Mistake-aware remediation using the wrong option chosen. Support for descriptive and coding questions. An LMS (Moodle) plugin. Multilingual content.

Q: How much does it cost to run?

A: Nothing for a class-sized demo. All three providers are used on free tiers, and Ollama allows fully offline, free running.

G. About the implementation

Q: What technologies did you use?

A: Python, PyTorch (EKT and DKT), sentence-transformers (MiniLM), FAISS, FastAPI with an HTML/JS frontend, OpenAI-compatible clients for Groq, OpenRouter and Gemini, and pytest for testing.

Q: How did you test it?

A: 35 automated pytest tests cover chunking, concept extraction, JSON parsing, rate-limit retry, the circuit breaker, pacing, knowledge-tracing metrics and the full pipeline with the mock model.

Q: How do you change the AI model?

A: Edit config.yaml (primary, verifiers, model names). All providers share the same OpenAI-compatible interface, so no code change is needed.

Tips for the review

- Start with the problem (same teaching for everyone), then the loop, then the three AI roles.
- Draw the loop on the board: Syllabus, then Concepts, Mastery (EKT), Gaps, Plan, Questions, Test, and back to Mastery.
- Show the live demo: pick a weak student, show the gap, the remedial plan and the adaptive test.
- Say "one writes, two verify" every time the AI models come up. It is easy for mentors to remember.
- Say clearly that the student data is simulated, and give the real trial as future work.